MCU

ESP32-S3 module

File: mcu.kicad_sch

USB

USB ports and data-path muxes

File: usb.kicad_sch

DUT power & AUX

DUT power switch, monitor and AUX

File: dut.kicad_sch

Rack bus

Rack bus: CAN transceiver and POS addressing token

File: bus.kicad_sch

DATA_INT, ESP_DN/DP, I2C_SCL/SDA, MUX_FORCE_PC, PC_DET, SVC

DUT_5V, I2C_SCL/SDA

POS_OUT

AUX_BOOT_CTL, AUX_RST_CTL, AUX_RX/TX, DUT_EN, DUT_FLT_N, I2C_SCL/SDA, INA_ALERT_N, POS_OUT

CAN_RXD/TXD, POS_IN/OUT

CAN_H/L, POS_IN_CONN, POS_OUT_CONN

CAM_CS/LED/SCK, CAM_MISO, CAM_MOSI

Bay node — one per bay. Sheets:
Power: row cable (JST PH 6: bus, GND, CAN_H, CAN_L, GND, POS), reverse protection, LMR51625 buck, AP2114H
MCU: ESP32-S3-WROOM-1-N8R8 (8 MB flash, 8 MB octal PSRAM), reset/boot, status LED, UART pads
USB: front PC port, the DUT's USB on the DATA lead (J7), two FSUSB42 muxes + OR gate (classic / USB-IP / service modes)
DUT power & AUX: TPS2553D switch → shunt → DUT_5V, INA226 monitor, AUX UART/RESET/BOOT
Camera: JST PH 8 to the bay's Arducam Mega (SPI2, 5 V through a PTC) and its LEDs (AO3400A low side)
Rack bus: TJA1462AT CAN transceiver on the row pair, POS token in/out (docs/can.md)

Mode truth table: SERVICE=0, PC unplugged → DUT↔ESP32 (USB/IP)
SERVICE=0, PC plugged → DUT↔PC (classic), ESP32 still controls DUT_5V/AUX
SERVICE=1 → PC↔ESP32 (flash/console), DUT data disconnected
GPIO firmware can also force classic mode (MUX_FORCE_PC).

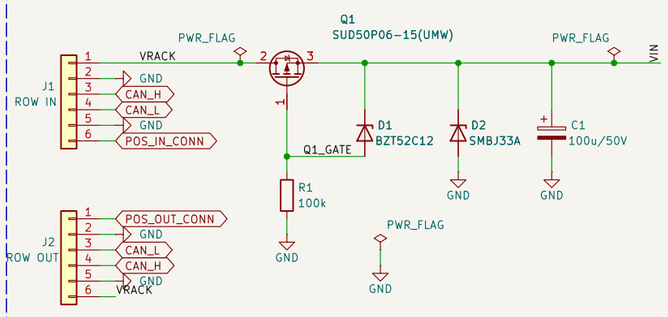
Row bus 20-28 V + CAN + POS in/out · 5 V/1.4 A switched DUT power on the DATA lead · USB 2.0 n
Generated from electronics/bay-node/design.py — edit the Python, then regenerate
ESP32-S3 USB/IP host for one bay of the hw-testing-rack shelf
hw-testing-rack

Sheet: /
File: bay-node.kicad_sch

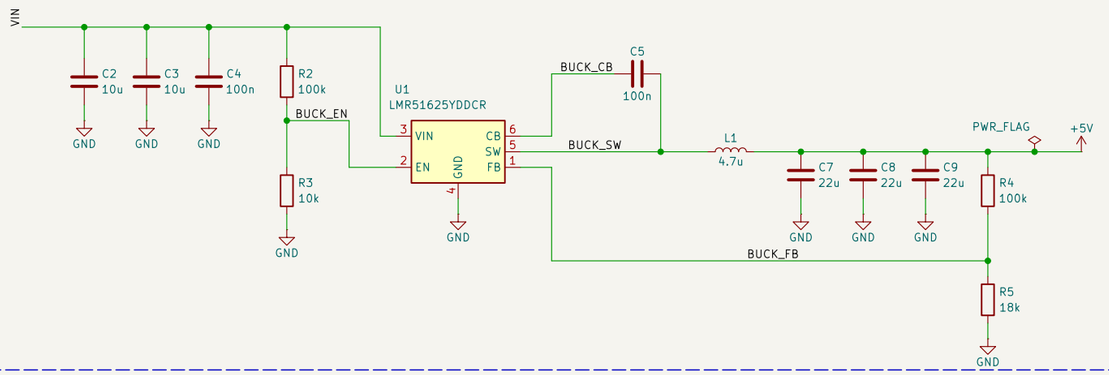
Title: Bay node — Overview

Size: A3	Date:	**Rev: 0.5**
KiCad E.D.A. 10.0.6		Id: 1/7

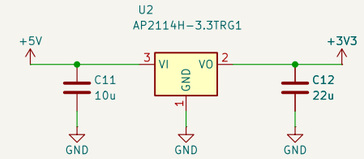
Row cable in / out (JST PH 6, daisy chain: bus, GND, CAN_H, CAN_L, GND, POS) and input protection



5 V buck: LMR51625, synchronous, 1.1 MHz, 2.5 A (TI SLUSF64B typical application)



3.3 V logic supply



Supply test pads (bottom side, labelled): no short to GND, then each rail at power-up



LMR51625YDDCR (2.5 A; a bay draws ≤ 2.25 A): 1.1 MHz, PFM at light load, spread spectrum. TI SLUSF64B table 8-1 (5 V, 1.1 MHz):
L 4.7 µH (FXL0630, 7.5 A sat guaranteed, 9 A typ); COUT 3 × 22 µF/25 V (≥ 33 µF left after DC-bias derating); CIN 2 × 10 µF/50 V + 100 nF; CB 100 nF.
At 2.25 A from 28 V the inductor peaks at ≈ 2.65 A (0.8 A ripple), under the 2.9 A minimum peak current limit.
FB 100k/18k: VOUT = 0.8 V × (1 + 100/18) = 5.24 V: ≈ 4.75 V still reaches the DATA board at 1.4 A over 50 cm
of PH lead (switch, shunt, one PH pin each way); everything on +5V takes it (TJA1462AT ≤ 5.5 V).
EN 100k/10k: the buck starts above 13.5 V and stops below 10.8 V (VEN 1.226 V, 0.241 V hysteresis), so no node
draws its low-voltage inrush while the hub's eFuse is soft-starting the row.
Layout: C4/C2/C3 straddle U1's VIN and GND pins on the top layer; CIN, U1 and COUT share one top-side GND island
with vias to the In1 plane; the FB divider sits at the FB pin, its +5V sense comes from COUT on another layer.
Input: SUD50P06 P-FET (reverse polarity, Vgs clamped by the 12 V zener); SMBJ33A clamps hot-plug ringing below 54 V (U1: 63 V abs max).
VRACK is 20 V, or 28 V (EPR), from the hub's USB-C PD supply: SMBJ33A stands off 33 V (28 V + 5 % = 29.4 V), the buck runs
4.3-60 V and starts above 13.5 V, the P-FET and the zener clamp are rated 60 V / 12 V. At 20 V a row draws more current: ≤ 1.6 A.

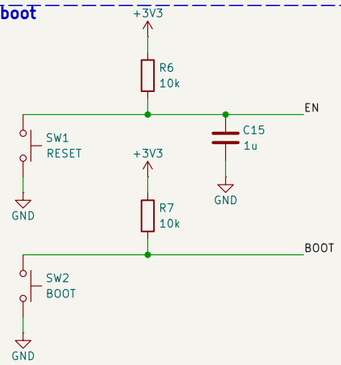
Row bus 20-28 V + CAN + POS in/out · 5 V/1.4 A switched DUT power on the DATA lead · USB 2.0 r
Generated from electronics/bay-node/design.py — edit the Python, then regenerate
ESP32-S3 USB/IP host for one bay of the hw-testing-rack shelf
hw-testing-rack

Sheet: /Power/
File: power.kicad_sch

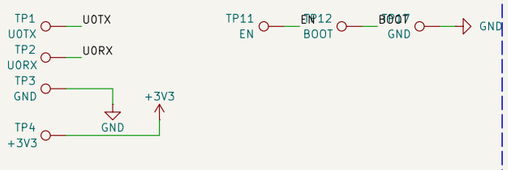
Title: Bay node — VRACK input (20-28 V), 5 V buck, 3.3 V LDO

Size: A3	Date:	**Rev: 0.5**
KiCad E.D.A. 10.0.6		Id: 2/7

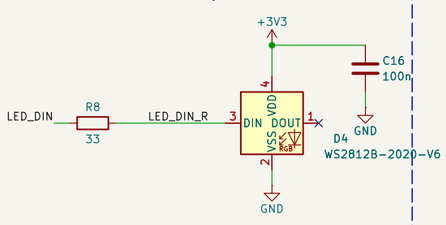
Reset and boot



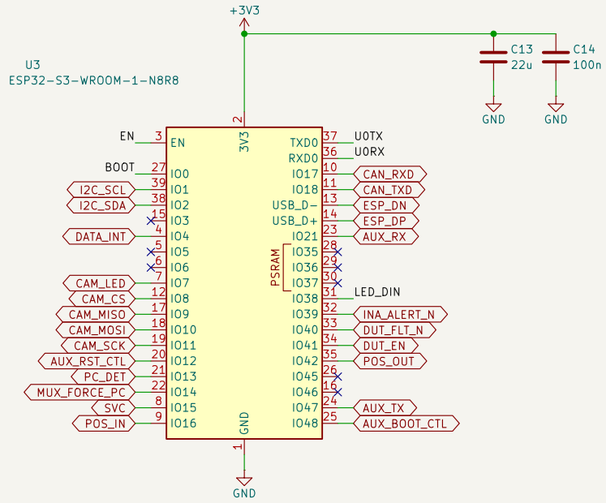
Debug test pads (bottom side): UART, EN, IO0, GND, 3.3 V



Status LED (behind the fascia window)



ESP32-S3-WROOM-1-N8R8 (PCB antenna at the board's rear edge: no copper under it, any layer)



GPIO map: IO41 DUT_EN · IO40 DUT_FLT_N · IO39 INA_ALERT · IO2/1 I2C SDA/SCL · IO47/21 UART1 TX/RX (AUX) ·
IO12 AUX_RST · IO48 AUX_BOOT · IO38 LED · IO13 PC_DET · IO14 MUX_FORCE_PC · IO15 SVC · IO19/20 USB (OTG/JTAG) ·
IO18/17 CAN TX/RX (TWAI) · IO16 POS_IN · IO42 POS_OUT · camera SPI2: IO11 SCK · IO10 MOSI · IO9 MISO · IO8 CS ·
IO7 light · IO4 DATA_INT (J7.8, open drain, active low). Free: IO5, IO6. IO35-IO37 are the octal PSRAM's;
the WROOM-1 has no IO26/IO33/IO34. The I²C bus (IO1/IO2) also goes out on J7.6/7 to the drawer's DATA board.
The map is the ESP32-S3-DevKitC-1 v1.1's (N8R8) as is: the bench stand-in needs no remap.
Strapping pins IO0/IO3/IO45/IO46 are not used for I/O. USB is full-speed only (one internal PHY).

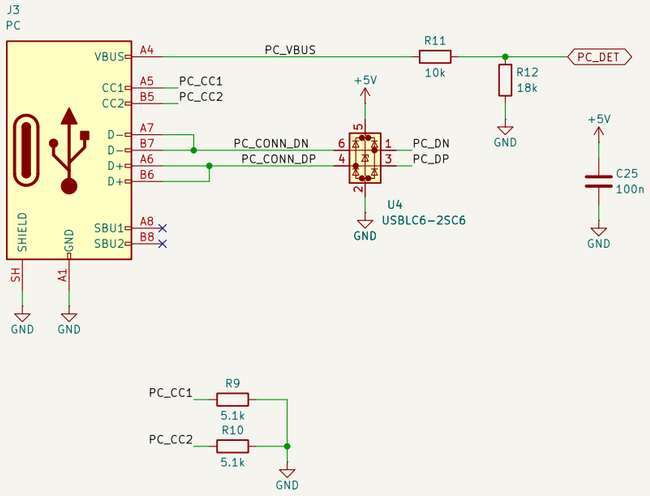
Row bus 20-28 V + CAN + POS in/out · 5 V/1.4 A switched DUT power on the DATA lead · USB 2.0 n
Generated from electronics/bay-node/design.py — edit the Python, then regenerate
ESP32-S3 USB/IP host for one bay of the hw-testing-rack shelf
hw-testing-rack

Sheet: /MCU/
File: mcu.kicad_sch

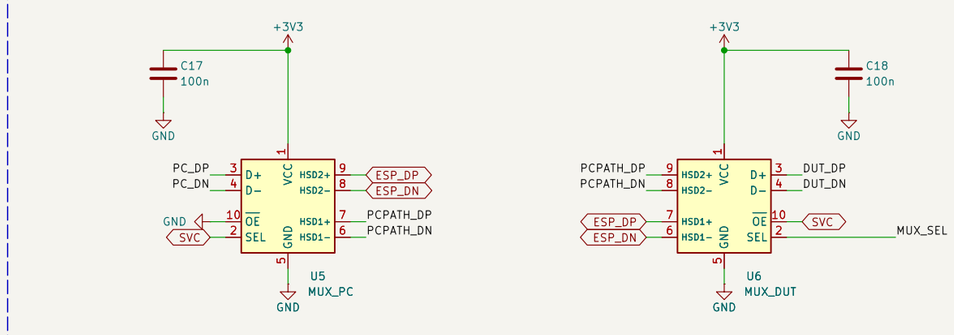
Title: Bay node — ESP32-S3 module

Size: A3	Date:	**Rev: 0.5**
KiCad E.D.A. 10.0.6		Id: 3/7

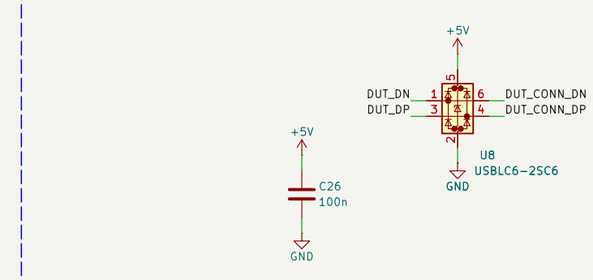
Front PC port: USB-C sink (Rd 5.1k on CC), VBUS only sensed



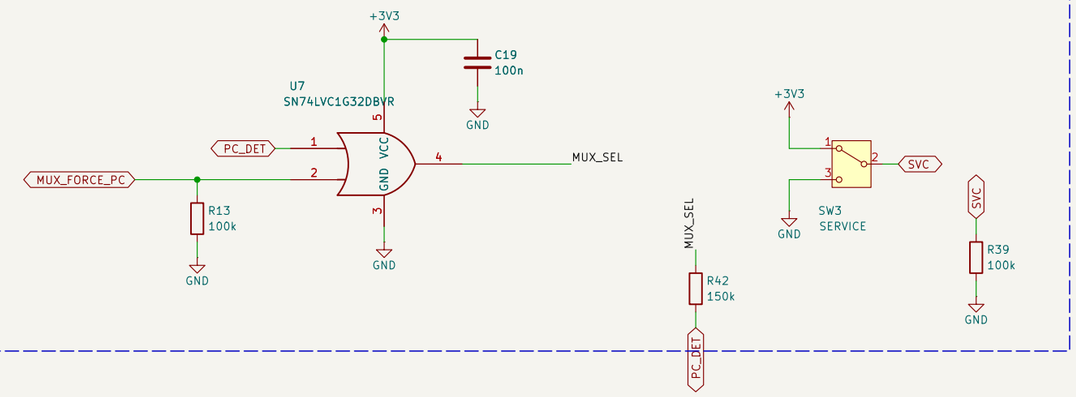
USB 2.0 data muxes (FSUSB42): MUX_PC → DUT path or ESP32, MUX_DUT → ESP32 or PC path



DUT USB: ESD at the DATA lead's node end (flow-through), towards J7

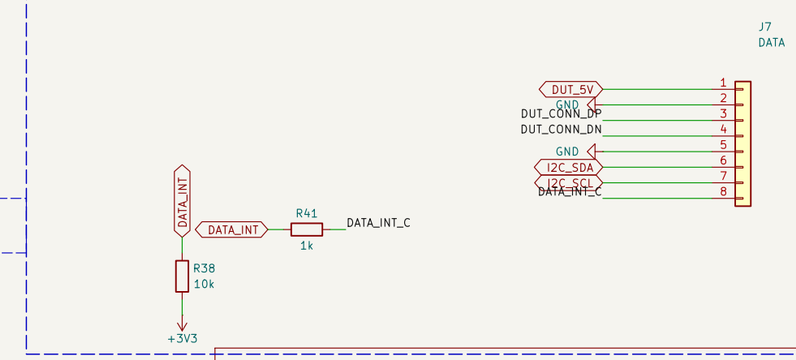


Mode logic: MUX_SEL = PC_DET OR MUX_FORCE_PC; SERVICE switch



SVC=1 routes the PC port to the ESP32 (flash / console / JTAG) and disables MUX_DUT (-OE high).
MUX_SEL = PC_DET OR MUX_FORCE_PC: a laptop on the front port takes the DUT over automatically.

DATA: the DUT's USB, I²C and INT on an 8-wire PH lead to the drawer's DATA board



Row bus 20-28 V + CAN + POS in/out · 5 V/1.4 A switched DUT power on the DATA lead · USB 2.0 n
Generated from electronics/bay-node/design.py — edit the Python, then regenerate
ESP32-S3 USB/IP host for one bay of the hw-testing-rack shelf

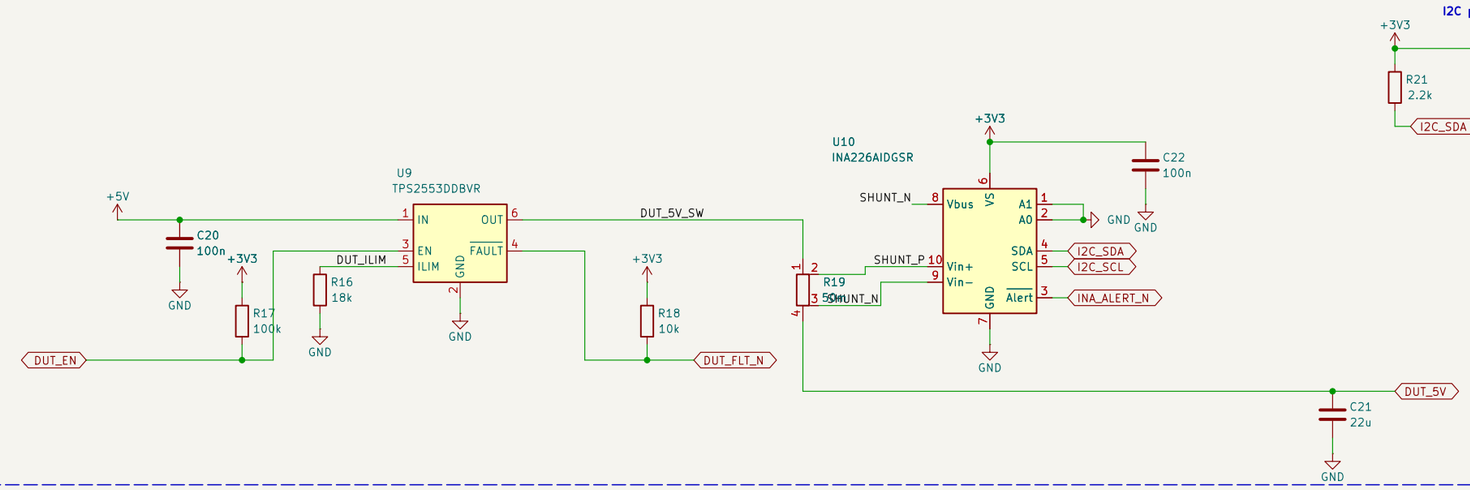
hw-testing-rack

Sheet: /USB/
File: usb.kicad_sch

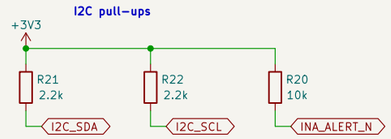
Title: Bay node — USB ports and data-path muxes

Size: A3	Date:	**Rev: 0.5**
KiCad E.D.A. 10.0.6		Id: 4/7

DUT power path: +5V → TPS2553D switch → 50 mΩ shunt → DUT_5V (J7.1), INA226 across the shunt

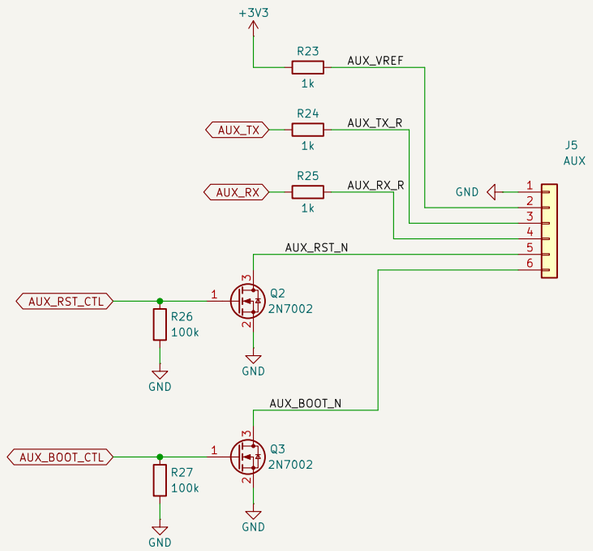


I2C pull-ups



TPS2553: ILIM 18k → 1.34–1.52 A. EN is pulled up: DUT_5V is on even with no firmware; -FAULT flags current limit / over-temperature.
INA226 (I2C 0x40) runs from +3V3: it measures the drop across R19 (50 mΩ → 1.64 A full scale, Kelvin sense pads); Vbus on Vin−: DUT_5V at the shunt.
AUX: TX/RX through 1k, VREF = +3V3 through 1k (R23, a level hint), RESET/BOOT open-drain (2N7002, 100k gate pull-downs).

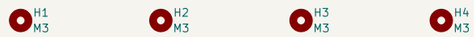
AUX: UART + open-drain RESET / BOOT towards the DUT (JST PH 6)



Test pads (bottom side): DUT_5V in the supply row; faults and I²C in the debug block



Mounting (M3, matches the carrier standoffs)



Test pads, second round (bottom side): AUX UART, RESET/BOOT, POS_OUT, a GND



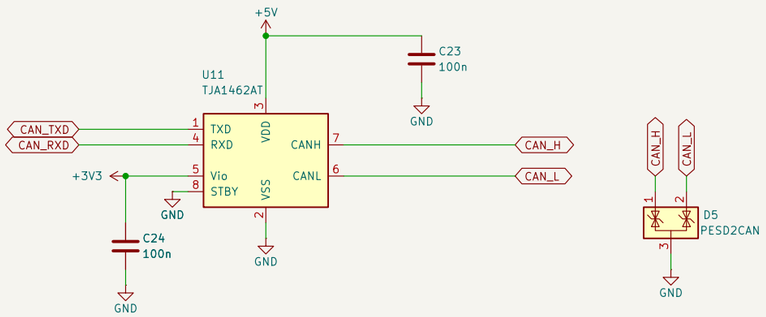
Row bus 20-28 V + CAN + POS in/out · 5 V/1.4 A switched DUT power on the DATA lead · USB 2.0 r
Generated from electronics/bay-node/design.py — edit the Python, then regenerate
ESP32-S3 USB/IP host for one bay of the hw-testing-rack shelf
hw-testing-rack

Sheet: /DUT power & AUX/
File: dut.kicad_sch

Title: Bay node — DUT power switch, monitor and AUX

Size: A3	Date:	**Rev: 0.5**
KiCad E.D.A. 10.0.6		Id: 5/7

CAN: TJA1462AT on the row pair (J1.3/4 ↔ J2.4/3, Power sheet); stub ≤ 20 mm, no termination

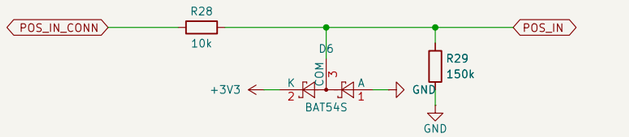


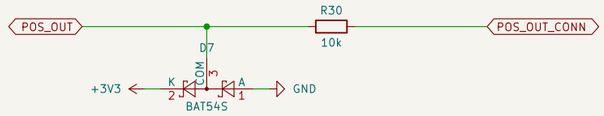
STB low: always in normal mode (TXD dominant time-out guards the bus).

Test pads, second round (bottom side): CAN TXD/RXD, POS_IN, a GND



POS token: in from J1.6, out to J2.1; 10k series + BAT54S clamp on each (VRACK on it costs ≈ 2.5 mA at 28 V)





CAN (docs/can.md): one bus for the rack, a star from the hub; both terminations sit in the hub.
250 kbit/s (125 kbit/s fallback). TJA1462AT (SIC): VCC 5 V, VIO 3.3 V, bus pins −36/+40 V, passive unpowered.
POS: active-high 3.3 V token. From reset the node holds POS_OUT low; when POS_IN goes high and it has no address yet, it answers with its MAC, takes (row, position) from the hub, then raises POS_OUT.
R29 keeps POS_IN low with no cable; the next node's 150k keeps POS_OUT low before firmware.

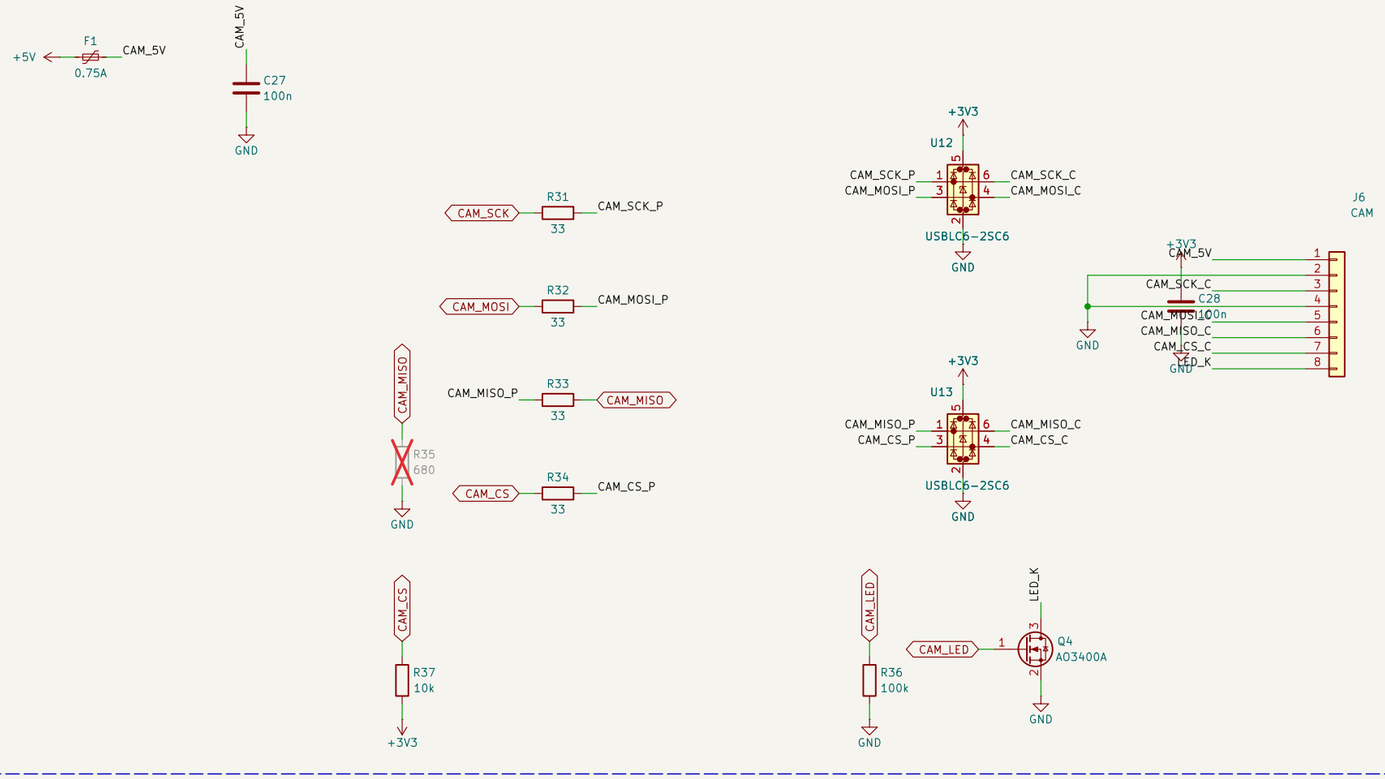
Row bus 20-28 V + CAN + POS in/out · 5 V/1.4 A switched DUT power on the DATA lead · USB 2.0 r
Generated from electronics/bay-node/design.py — edit the Python, then regenerate
ESP32-S3 USB/IP host for one bay of the hw-testing-rack shelf
hw-testing-rack

Sheet: /Rack bus/
File: bus.kicad_sch

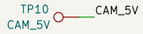
Title: Bay node — Rack bus: CAN transceiver and POS addressing token

Size: A3	Date:	**Rev: 0.5**
KiCad E.D.A. 10.0.6		Id: 6/7

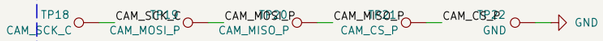
J6: JST PH 8 to the bay's camera pod (Arducam Mega 3MP SPI + white LEDs), ≈ 50 cm



CAM_5V test pad (bottom side, in the supply row)



Test pads, second round (bottom side): the camera SPI and a GND



Lead pinout: 1 CAM_5V · 2 GND · 3 SCK · 4 GND · 5 MOSI · 6 MISO · 7 CS · 8 LED_K (the LEDs' cathode side).
SPI2 through the GPIO matrix, mode 0, 4 MHz (2 MHz fallback): IO11 SCK, IO10 MOSI, IO9 MISO, IO8 CS (10k
  pull-up: deselected before firmware), IO7 the light (R36 holds it off at boot; LEDC PWM ≥ 20 kHz or on/off).
CAM_5V through F1 (0.75 A PTC, 1.5 A trip): the camera ≤ 317 mA (Arducam) + the LEDs ≤ 102 mA. USBLC6s on the lead side of the 33 Ω.
The camera pod board runs the Mega from 3.3 V (its own AP2114H): MISO is 3.3 V and R35 stays unpopulated.
  For a camera fed 5 V that drives MISO at 5 V instead: R33 → 330 Ω and fit R35 (680 Ω): 3.37 V.

Row bus 20-28 V + CAN + POS in/out · 5 V/1.4 A switched DUT power on the DATA lead · USB 2.0 r
Generated from electronics/bay-node/design.py — edit the Python, then regenerate
ESP32-S3 USB/IP host for one bay of the hw-testing-rack shelf
hw-testing-rack

Sheet: /Camera/
File: cam.kicad_sch

Title: Bay node — Camera lead: Arducam Mega SPI and its LEDs

Size: A3	Date:	**Rev: 0.5**
KiCad E.D.A. 10.0.6		Id: 7/7